UNIVERSITY OF THE PEOPLE

Dynamic CS Vocabulary Engine

An Adaptive True/False Vocabulary Trainer for Introductory Computer Science

A Capstone Project Report of MSIT 5910

Submitted by:

Roger Rios

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Sirisha Pavuluri

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

Term 1, October 2026

UNIVERSITY OF THE PEOPLE

ABSTRACT

Introductory computer science classes mix students who learn at very different rates, and fixed-difficulty practice leaves beginners overloaded and advanced students bored. This project designed, built, and evaluated the Dynamic CS Vocabulary Engine, a True/False trainer for programming vocabulary that adapts to each student. It is a single Python file written with the CMU Graphics package and runs in the CMU CS Academy browser sandbox, so a school can use it with nothing to install. A rule-based dynamic difficulty adjustment (DDA) engine moves the student among three tiers of terms using two measures any program can record: answer time and errors. The decision logic was written as pure functions and tested outside the browser with stand-ins for the graphics objects. Release v0.4.1 passes 80 automated tests that cover every executable line and detect 30 of 30 planted faults. In system testing in the sandbox it passed all 17 test cases, updated the display a median of 54.8 ms after a key press, and kept a three-minute session within 0.09% of its length. Eight of nine requirements were accepted; a manual override for teachers remains unbuilt. Simulation showed that the engine serves lower-tier learners questions at their own level only about half the time, because its promotion rules react to single answers. The engineering is therefore verified, while the educational benefit is untested: no students have used the tool. The report also sets out a maintenance plan and proposes steadier promotion rules and a classroom trial.

UNIVERSITY OF THE PEOPLE

ACKNOWLEDGEMENT

I thank my instructor, Sirisha Pavuluri, for her guidance and feedback throughout the capstone course. I thank the chair and faculty of the Department of Computer Science and MSIT for a programme that prepared me for this project, and University of the People for making a tuition-free graduate education possible. I am grateful to the CMU CS Academy team, whose free curriculum and sandbox made a browser-based tool practical for schools.

The code was AI-assisted, specifically the DDA algorithm and the corrections. I reviewed and tested all of it, and I take responsibility for it.

UNIVERSITY OF THE PEOPLE

LIST OF TABLES

Table 1.1. Changes from the original proposal....12
Table 2.1. Comparison of approaches to adaptive practice....17
Table 3.1. DDA rules in the order they are checked....21
Table 3.2. Functional and non-functional requirements....22
Table 3.3. Modules of main.py (release v0.4.1)....24
Table 4.1. Implementation milestones by unit....26
Table 4.2. Input–output log of the recorded session....29
Table 4.3. Test levels and results....34
Table 4.4. Performance and reliability results....38
Table 4.5. Simulated learners: questions served relative to the learner's tier....39
Table 4.6. Heuristic review findings and their status in release v0.4.1....40
Table 5.1. Requirements and their status in release v0.4.1....44
Table 5.2. Maintenance plan....46
Table A.1. System test cases run in the CS Academy sandbox on release v0.4.1....54

UNIVERSITY OF THE PEOPLE

LIST OF FIGURES

Figure 3.1. System architecture and data flow....19
Figure 3.2. Development and release workflow....20
Figure 3.3. The DDA engine (main.py, lines 116–139)....21
Figure 4.1. Helper functions added or changed in release v0.4.1 (main.py, lines 158–184)....27
Figure 4.2. The system in the CS Academy sandbox....28
Figure 4.3. Wrong key, streak promotion, and overload demotion....30
Figure 4.4. Difficulty tier after each answer in the recorded session....31
Figure 4.5. Long prompts drawn in full on two lines....32
Figure 4.6. End-of-session report (a); invalid keys ignored (b)....33
Figure 4.7. Failed test cases before the fixes and before the test update....34
Figure 4.8. Integration tests and the full run with line coverage....35
Figure 4.9. System test before (v0.3.0) and after (v0.4.1) the fixes....36
Figure 4.10. Output of the acceptance checklist....37
Figure 4.11. Timing recorded in the CS Academy sandbox....38
Figure 5.1. Key press to updated display for all 36 measured presses....41
Figure 5.2. Simulated sessions: questions at the learner’s tier, by version....42
Figure B.1. State manager: loadScenario() and applyDDA() (lines 200–234)....55
Figure B.2. Event listeners: onStep() and onKeyPress() (lines 251–284)....55
Figure C.1. Mutation check: 30 of 30 planted faults detected....56
Figure C.2. Engine benchmark output....56
Figure D.1. Gantt chart of the project by course unit (3 September to 28 October 2026)....57

UNIVERSITY OF THE PEOPLE

SYMBOLS AND ABBREVIATIONS

Term	Meaning
AI	Artificial intelligence
CS	Computer science
DDA	Dynamic difficulty adjustment
FR	Functional requirement
KiB	Kibibyte (1,024 bytes)
ms, µs	Millisecond, microsecond
NFR	Non-functional requirement
SHA-256	Secure Hash Algorithm, 256-bit digest
STEM	Science, technology, engineering, and mathematics

UNIVERSITY OF THE PEOPLE

CONTENTS

Abstract 2
Acknowledgement 3
List of Tables 4
List of Figures 5
Symbols and Abbreviations 6
1. Introduction 9
1.1 Background of the Study 9
1.2 Problem Statement 10
1.3 Objectives of the Study 10
1.4 Scope and Limitations 11
1.5 Report Organization 12
2. Literature Review 13
2.1 Overview of Related Work 13
2.1.1 Cognitive Load and Computer Science Pedagogy 13
2.1.2 Dynamic Difficulty Adjustment in Serious Games 13
2.1.3 Game Elements in a Minimal Interface 14
2.1.4 Requirements, Architecture, and Version Control 14
2.1.5 Testing, Quality, and Maintenance 15
2.1.6 AI-Assisted Development 16
2.2 Comparative Analysis of Existing Systems 16
2.3 Research Gap Identification 17
3. System Design and Methodology 18
3.1 System Architecture 18
3.2 Methodologies and Tools Used 20
3.2.1 Development Process and Tools 20
3.2.2 The DDA Algorithm 20
3.2.3 Requirements 22
3.2.4 Testing and Evaluation Method 22
3.2.5 Ethical and Security Considerations 23
3.3 Data Flow and System Components 24
3.4 Design Diagrams 24
4. Results and Evaluation 26
4.1 Implementation Overview 26
4.2 Output and Screenshots 27
4.3 Evaluation Metrics 33

4.3.1 Correctness and Regression....33
4.3.2 System and Acceptance Testing....35
4.3.3 Performance and Reliability....37
4.3.4 Adaptation Quality....38
4.3.5 Usability....39
5. Discussion....41
5.1 Interpretation of Results....41
5.2 Comparison with Literature Findings....43
5.3 Implications and Limitations....44
5.4 Future Work....46
6. Conclusion and Future Work....48
6.1 Summary of Main Findings....48
6.2 Contributions....48
6.3 Lessons Learned....49
6.4 Next Steps....49
References....50
Appendices....54
Appendix A: System Test Cases....54
Appendix B: Code Listings....55
Appendix C: Evaluation Output....56
Appendix D: Project Schedule....57

1. INTRODUCTION

1.1 Background of the Study

Introductory computer science (CS) in secondary school asks students to learn two things at once: how to think in steps, loops, and conditions, and the vocabulary in which those ideas are taught. Terms such as variable, parameter, and return are the working language of every later lesson, and a student who is unsure of them struggles to follow instructions or to ask for help. Practice material for this vocabulary is usually static. Worksheets, quizzes, and automatic graders present the same items in the same order to every student and report a pass or a fail.

A single classroom, however, contains students who learn at very different rates. Fixed-difficulty practice therefore fails at both ends: advanced students are bored because nothing challenges them, and beginners are overwhelmed when the difficulty moves faster than their understanding. Games have long addressed the same problem with dynamic difficulty adjustment (DDA), which changes the challenge while the player plays. A recent review of DDA in serious games found that most of the studies it covered reported successful or promising results (Víteková et al., 2026), and Seyderhelm and Blackmore (2024) found that an adaptive training game reached similar performance to a linear one with lower cognitive load.

This project applies that idea to vocabulary practice under the constraints of an ordinary school. Many schools issue managed Chromebooks on which students cannot install software, so the tool has to run in a browser. It was built with the CMU Graphics package and runs in the CMU CS Academy sandbox, a free environment for beginner Python courses.

1.2 Problem Statement

High school students studying computer science often fail to stay engaged in their first programming units, because linear, fixed-difficulty practice does not adjust to the rate at which each student learns. It is therefore necessary to develop a browser-based practice tool, written in Python with the CMU Graphics package, that uses dynamic difficulty adjustment to keep students engaged, reduce their anxiety, and help them master the basic vocabulary of programming.

Three groups are affected. Students are the primary users and bear the cost of static difficulty directly. Teachers cannot differentiate practice by hand for thirty students progressing at thirty rates, and an adaptive tool frees them to work with the students who need personal attention. School administrators work under tight budgets, and a tool that runs in the browser on devices the school already owns, with no installation and no licence fee, supports STEM learning without new spending.

1.3 Objectives of the Study

The aim was to deliver a working, evaluated prototype within the eight-week capstone period. The objectives were:

1. To build a True/False vocabulary trainer in Python with CMU Graphics that runs in the CS Academy sandbox in a browser, with school Chromebooks as the target device.
2. To implement a DDA engine that raises, holds, or lowers the difficulty tier from two performance measures, answer time and errors.
3. To cover every rule of the engine with an automated test and to respond to a key press in under one second.

4. To evaluate correctness, performance, adaptation quality, and usability with quantitative and qualitative metrics.
5. To test the complete system at integration, system, and acceptance levels and to plan its deployment and maintenance.
6. To keep the code, tests, and documentation in a public, version-controlled repository.

1.4 Scope and Limitations

The project covers the design, coding, testing, and evaluation of the trainer: 23 terms in three tiers (basics, control flow, and functions and scope), a real-time DDA engine, and a three-minute session that ends with a short report. It does not include typed code answers, mouse or touch input, sound, animation, multiplayer features, user accounts, stored student records, or a teacher dashboard. It assumes that the student has a computer with a current browser, a keyboard, internet access, and a CS Academy account. CS Academy runs one file at a time, so the application had to fit in a single Python file.

The scope changed once. The proposal of 16 September described a terminal program in which students typed lines of code, with input sanitization and encrypted progress files. Students cannot run Python scripts on managed Chromebooks, so on 30 September the project moved to CMU Graphics in the browser. Table 1.1 records what changed. The problem, the stakeholders, and the adaptive approach did not.

Table 1.1. Changes from the original proposal.

Aspect	Proposed	Built
Platform	Python script in a terminal	CMU Graphics in the CS Academy sandbox
Student input	Typed text and code	Two keys: Y for True, N for False
Content	Logic puzzles	23 vocabulary terms in three tiers
Data handling	Encrypted local progress files	Nothing stored; a report at the end
Development tools	Visual Studio Code	Google Antigravity with Google Gemini
Testing	Manual, in week 7	Automated with pytest from week 5

The main limitation is that no students have used the system. Every behavioural result in this report comes from scripted sessions, simulation, or my own testing, so the evaluation verifies the software without validating its effect on learning. Section 5.3 lists the other limitations.

1.5 Report Organization

Chapter 2 reviews the literature on cognitive load, DDA, and the software engineering practices the project relies on, compares existing approaches, and identifies the gap. Chapter 3 presents the architecture, the DDA algorithm, the requirements, and the tools and methods. Chapter 4 reports the implementation, the outputs of the running system, and the evaluation, including the system and acceptance testing of release v0.4.1. Chapter 5 interprets the results, compares them with the literature, and discusses deployment, maintenance, limitations, and future research. Chapter 6 concludes. The appendices hold the system test cases, code listings, evaluation output, and schedule.

2. LITERATURE REVIEW

The project draws on educational psychology, adaptive game design, and software engineering. This chapter reviews each, compares existing approaches to practice in introductory CS, and identifies the gap the project addresses.

2.1 Overview of Related Work

2.1.1 Cognitive Load and Computer Science Pedagogy

Cognitive Load Theory holds that the capacity and duration of working memory are very limited. A student learning to program has to handle the syntax of the language, the logic of the algorithm, and the vocabulary in which both are taught, and this combined demand can exceed working memory and cause overload. Seyderhelm and Blackmore (2024) studied an adaptive training game that adjusted its difficulty from real-time measures of cognitive load and performance. Compared with a linear version of the same game, the adaptive version reached similar performance with lower cognitive load and in less time. This project had no practical way to measure cognitive load in a classroom, so it relies on two indirect indicators that any program can record: the time taken to answer and the number of errors. The interface is also kept plain, with one screen of text labels and no animation or sound, to limit the extraneous load of a distracting presentation.

2.1.2 Dynamic Difficulty Adjustment in Serious Games

DDA comes from video game design. A DDA system watches how the player is performing and adjusts the difficulty as play continues, with the aim of keeping the player in a state of flow, in which challenge matches ability and both boredom and anxiety are avoided (Víteková et al., 2026). Víteková et al. reviewed 75 studies of DDA in serious games published between 2020 and 2025. Their starting point is that a fixed difficulty curve cannot suit users who learn at different rates.

Rule-based and heuristic methods were the most common approach in their sample, and most systems modelled the player from in-game performance measures, such as task accuracy, that can be tracked without interrupting play. Most of the studies reported successful or promising results, although the authors caution that outcome measures differ widely between studies and that many samples are small. Only 12 of the 75 papers reported no evaluation of effectiveness, so some evaluation is the norm in the field. In this project a rule-based engine acts as an automatic tutor: it raises the tier of vocabulary for students who answer quickly and correctly and lowers it for students who are slow or who make repeated mistakes.

2.1.3 Game Elements in a Minimal Interface

The original proposal argued for a text-only terminal on the grounds that visual noise distracts from learning. The move to a graphical platform keeps that principle. The design assumes that engagement in a learning game depends less on visual richness than on the loop of challenge, feedback, and reward, and the interface uses only the game elements that serve that loop. Feedback is immediate: a wrong key brings up a one-sentence correction, and a right key brings the next term. Nielsen (1993) gives 0.1 second as the limit under which a system feels as if it reacts instantaneously, and that is the target for both responses. Progress is visible through a tier label and a streak counter, and a countdown sets a short, fixed session. Points, badges, leaderboards, animation, and sound are left out.

2.1.4 Requirements, Architecture, and Version Control

Turning the idea into software needs an engineering framework. Summers (2020) stresses that effective software engineering depends on clearly defining and managing requirements, and Letaw (2024) separates functional requirements, which describe what a system does, from non-functional

requirements, which concern attributes such as performance, security, and usability. For a tool used by students the second kind matters as much as the first. Walker (2022) surveys architectural patterns and describes the layered pattern, which separates the user interface, the application logic, and the data so that one can change without breaking the others. Davies (2024) treats Git as essential for managing versions and project history, with branching and remote hosting as the means of protecting a stable version from experimental work. Configuration management extends this to everything that makes up a release. Bennett (2024) describes it as a process for systematically managing and controlling changes, built on configuration identification, baselines, change control, status accounting, and audits.

2.1.5 Testing, Quality, and Maintenance

Testing proceeds in levels. A unit test checks one piece of code in isolation, with stubs in place of real dependencies (Amazon Web Services, n.d.), and unit tests cannot detect integration or usability problems (GeeksforGeeks, 2026). System testing checks the complete, integrated application against its requirements, after integration testing and before acceptance testing (GeeksforGeeks, 2025b; Hrupa, 2024). BrowserStack (2025) describes its sequence: prepare an environment, write cases and data, execute, report defects, run regression tests, and retest each fix. Quality itself has to be measured. LaunchNotes (2023) distinguishes quantitative metrics, which are numerical and comparable, from qualitative metrics, which capture context that numbers miss, and GeeksforGeeks (2025a) lists correctness, performance, reliability, and usability among the attributes of software quality. For usability, Nielsen's (2024) ten heuristics and the contrast minimum of the World Wide Web Consortium (2026) give criteria that a single reviewer can apply.

Software also has to be maintained after release. Thales (n.d.) names four types of maintenance: corrective, which fixes faults; adaptive, which follows changes in technology; perfective, which adds or removes features; and preventive, which removes latent faults before they grow. GeeksforGeeks (2025c) adds that maintenance requires planning, testing, and version control.

2.1.6 AI-Assisted Development

The project was developed with the help of AI tools. Google Antigravity, the development environment, is described by its makers as an agentic development platform that combines a code editor with AI agents able to plan, carry out, and check multi-step tasks (Google Antigravity Team, 2025). In this project the code was AI-assisted, specifically the DDA algorithm and the corrections, with Google Gemini (Google, 2026) among the assistants used. Such tools shorten the path from an idea to a running prototype, but generated code can look correct and be wrong. Summers (2020) separates verification, which checks that a product is built to its specification, from validation, which checks that it meets the user's needs. This project treats AI-assisted code as unverified until it passes automated tests.

2.2 Comparative Analysis of Existing Systems

Table 2.1 compares three existing approaches with the one built here. Fixed-sequence practice is cheap and universal but ignores the learner. The adaptive system of Seyderhelm and Blackmore (2024) shows what adaptation can achieve when cognitive load is measured in real time, a measurement that an ordinary classroom cannot take. The rule-based serious games surveyed by Víteková et al. (2026) adapt from performance alone, as this project does, but they were evaluated in ways too varied to compare. None of the three gives a school a ready, free tool for programming vocabulary that runs on managed Chromebooks.

Table 2.1. Comparison of approaches to adaptive practice.

Approach	How difficulty is set	Signal used	Main limitation
Fixed-sequence practice (worksheets, quizzes, automatic graders)	Same order for every student	None; pass or fail	Does not adapt to the learner's rate
Adaptive training with measured cognitive load (Seyderhelm & Blackmore, 2024)	Adjusted in real time	Cognitive load and performance	Needs a real-time measure of load
Rule-based DDA in serious games (Víteková et al., 2026)	Rules or heuristics during play	In-game performance, such as accuracy	Evaluations small and inconsistent
This project	Four ordered rules after each answer	Answer time and errors	Not yet tested with students

2.3 Research Gap Identification

The literature leaves two gaps that this project addresses. The first is practical. DDA is well studied in serious games, but that research does not give a school a ready tool, and I did not find a lightweight, free, adaptive practice tool for programming vocabulary that runs in the single-file browser environment many schools already use. The second is methodological. Víteková et al. (2026) report that evaluation metrics for DDA are limited and inconsistent. A small rule-based engine offers a chance to evaluate adaptation transparently: every rule can be tested in isolation, and rule sets can be compared on simulated learners before students meet them. This project therefore asks whether a rule-based DDA engine can be built, fully tested, and measured within the constraints of a classroom sandbox, and how well simple time-and-error rules place learners at their level.

3. SYSTEM DESIGN AND METHODOLOGY

This chapter describes the architecture of the system, the algorithm at its centre, the requirements, and the tools and methods used to build and test it.

3.1 System Architecture

The system follows the layered pattern described by Walker (2022), with three layers (Figure 3.1). The interface layer draws the screen on a 400 × 400 CMU Graphics canvas and receives key presses and clock ticks. The screen has a header, eleven text labels (tier, question time, errors, streak, topic, term, two prompt lines, feedback, key instructions, and session countdown), a streak star, and a hidden report panel. The application logic layer holds the session state and the DDA rules. The data layer holds the vocabulary in memory, and nothing is written to disk. The layers are logical, not physical: CS Academy runs one file, so all three are sections of main.py, kept apart by function boundaries.

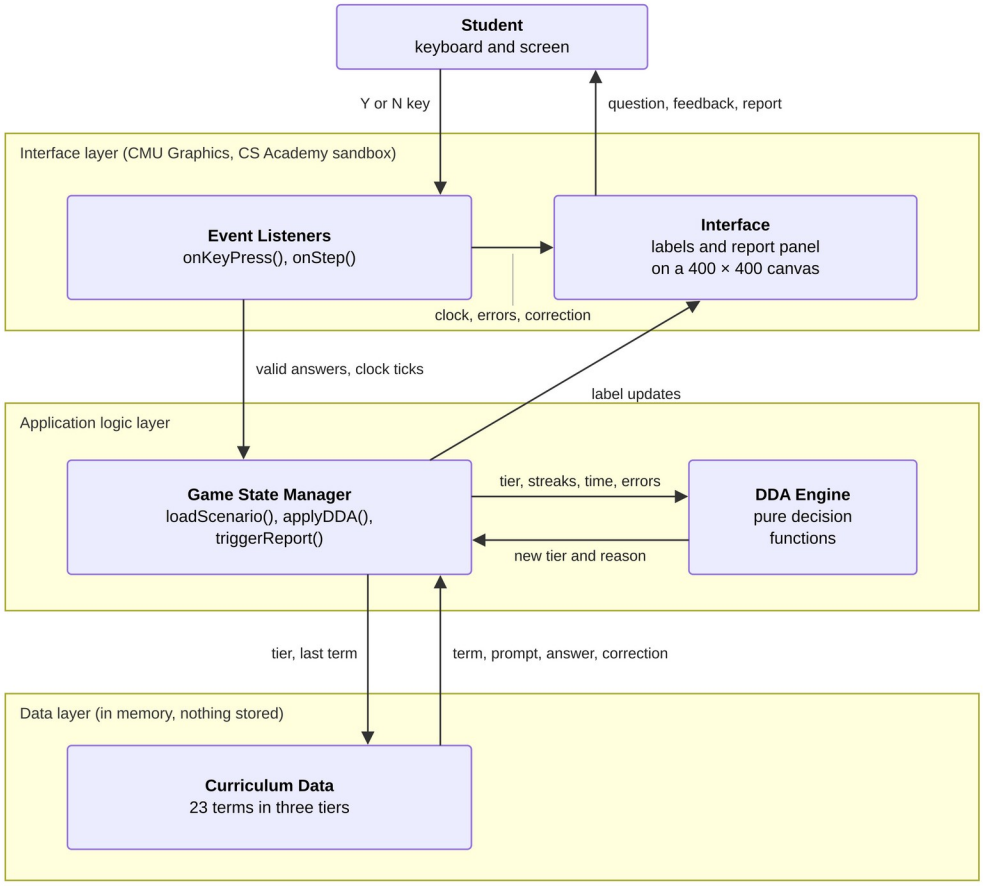


Figure 3.1. System architecture and data flow.

Two design decisions follow from the constraints. First, the DDA engine is a set of pure functions that make no graphics calls. CS Academy supplies its graphics objects as globals, so a single-file program cannot simply be imported into a test, but functions with defined inputs and outputs can be tested outside the browser. Second, input is limited to two keys. The original design had students type code, which required sanitization against code injection. With Y and N as the only accepted keys, and no use of eval() or exec(), that risk is designed out, and because nothing is stored there is no personal data to protect.

3.2 Methodologies and Tools Used

3.2.1 Development Process and Tools

Development was iterative, with one increment per course unit, planned on a Gantt chart for the eight weeks (Oguz, 2022; Appendix D). Code was edited in Google Antigravity, where the automated tests run, and was AI-assisted, specifically the DDA algorithm and the corrections. The language is Python 3 with the CMU Graphics package (CMU CS Academy, 2026), the tests use pytest, and the project is tracked with Git in a public GitHub repository (https://github.com/Rios1024/MSIT-Capstone). The main branch holds only code that has passed every test, work happens on dev, and each change starts on a short-lived feature branch. A GitHub Actions workflow runs the tests and the coverage report on every push and pull request; GitHub (n.d.) describes such a workflow as an automated process, defined in a file in the repository, that runs on a hosted runner. A release is a tagged commit on main whose main.py is pasted into the sandbox (Figure 3.2). Because that copy sits outside version control, it is compared with the tagged file by SHA-256 hash before students use it. AI-assisted code is merged only after it has been read, run, and passed by the same tests as hand-written code.

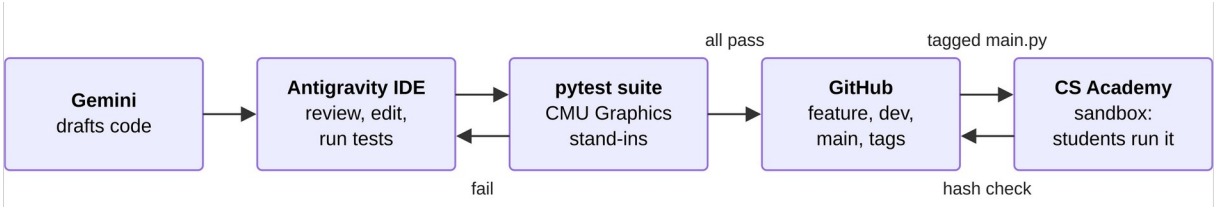


Figure 3.2. Development and release workflow.

3.2.2 The DDA Algorithm

After every completed question the engine receives five values: the current tier, the streak of consecutive first-try answers, the count of consecutive first-try misses, the seconds spent on the question, and the number of wrong key presses. It returns the next tier and the name of the rule that

decided it. Table 3.1 gives the four rules in the order they are checked, and Figure 3.3 shows the code. The order matters. Overload is checked first, so that a slow or missed answer can never raise the tier. Fast, error-free answers and sustained streaks raise the tier to keep the student in flow, with answer time and errors standing in for the cognitive load the program cannot measure. The time, streak, and miss thresholds are named constants at the top of the file, so a threshold can be changed in one reviewed line. The engine also records the reason for each decision, which supports the transparency promised in the proposal.

Table 3.1. DDA rules in the order they are checked.

Order	Rule	Trigger	Result
1	Overload	More than 15 s on the question, two wrong presses, or two first-try misses in a row	One tier down, not below Tier 1
2	Streak	Every third consecutive first-try answer	One tier up, not above Tier 3
3	Flow	First-try answer in under 5 s	One tier up
4	Steady	Anything else	Tier held

```
116 def updateStreaks(streak, missStreak, errors):
117     """Run once per completed question. Returns (streak, missStreak).
118     A first-try answer extends the streak; a corrected answer counts as a miss."""
119     if errors == 0:
120         return streak + 1, 0
121     return 0, missStreak + 1
122
123 def nextDifficulty(tier, streak, missStreak, seconds, errors):
124     """Decide the next tier. Returns (newTier, reason).
125     Overload is checked first so a slow or missed answer can never promote."""
126     # Rule 1: Cognitive Overload Mitigation
127     if seconds > SLOW_SECONDS or errors >= 2 or missStreak >= MISSES_TO_DEMOTE:
128         return max(MIN_TIER, tier - 1), 'overload'
129
130     # Rule 2: Promotion on every 3rd consecutive first-try answer
131     if streak > 0 and streak % STREAK_TO_PROMOTE == 0:
132         return min(MAX_TIER, tier + 1), 'streak'
133
134     # Rule 3: Flow State Recognition
135     if errors == 0 and seconds < FAST_SECONDS:
136         return min(MAX_TIER, tier + 1), 'flow'
137
138     # Rule 4: Optimal challenge, hold the tier
139     return tier, 'steady'
```

Figure 3.3. The DDA engine (main.py, lines 116–139).

3.2.3 Requirements

Table 3.2 lists the requirements, written in the manner described by Letaw (2024). They were restated when the platform changed: typed input became two keys (FR1), stored progress profiles became an end-of-session report (FR4), and sanitization and encryption gave way to a restricted input surface (NFR1). NFR5 was added so that content could grow without touching the engine.

Table 3.2. Functional and non-functional requirements.

ID	Requirement
FR1	Show one term and one True/False question at a time on a CMU Graphics canvas and accept answers through the Y and N keys only.
FR2	Raise, hold, or lower the tier of the next question from the time taken and the errors made on the current one.
FR3	Let a teacher set the tier by hand if the algorithm's judgment appears inaccurate.
FR4	Show a report at the end of each session and store nothing between sessions.
NFR1	Security: act only on the Y and N keys, never use eval() or exec(), read and write no files, and collect no personal data.
NFR2	Usability: every label fully visible on the canvas; text contrast of at least 4.5:1 (World Wide Web Consortium, 2026).
NFR3	Performance: the next question within one second of a key press, with a target of 0.1 second (Nielsen, 1993).
NFR4	Reliability: unexpected keys, and input after the session ends, are ignored without an error.
NFR5	Scalability: terms are data, so more terms or tiers can be added without changing the engine.

3.2.4 Testing and Evaluation Method

Quality assurance has five levels. Unit tests load main.py against lightweight stand-ins for the CS Academy objects and check each function, with boundary value analysis at the thresholds: an answer at 4.9 seconds against one at 5.0, and one at 15.0 against one at 15.1. Integration tests drive the program through onKeyPress() and onStep(), the two entry points the sandbox calls, and confirm after each answer that the tier, topic, error, streak, and term labels match the engine state. Usability regression tests check text width, contrast, the streak marker, and capital letters. System

tests run the released file in the real sandbox through 17 cases (Appendix A) while a script in the page time-stamps every key press and every change to the canvas. Acceptance testing is a scripted checklist with one criterion per requirement. It is run by the developer, and user acceptance testing with students is future work.

Three further methods evaluate what tests cannot. A mutation check plants one fault at a time and confirms that the tests notice. A simulation compares rule sets on synthetic learners. A heuristic review applies Nielsen's (2024) heuristics with his severity scale from 0 to 4 (Nielsen, 1994), supported by automated contrast and text-width checks.

3.2.5 Ethical and Security Considerations

Using an algorithm to judge students raises questions of fairness. Poorly calibrated rules may misjudge a student from unusual answer times, for instance by treating a careful reader as a struggling one, and on True/False questions a guess is right half the time, so luck can move a student up. The design responds with transparency: the main thresholds are named constants, the engine records the reason for every tier change, and tiers are practice levels that are never used as grades. A manual override for teachers (FR3) would complete this response and is not yet built. Privacy is handled by data minimization. The program collects no personally identifiable information, session statistics exist only in memory, and they are shown once, in the end-of-session report. Security rests on the restricted input surface described in Section 3.1 and on the browser sandbox, in which the program opens no files and makes no network requests. Finally, the code was AI-assisted, specifically the DDA algorithm and the corrections. That use is disclosed in this report, and every such change was read, run, and tested before it was kept.

3.3 Data Flow and System Components

Table 3.3 lists the five modules. The system has no network interface and no database, so the modules communicate through function calls and the shared app object. The path of one answer shows the data flow. A key press enters at onKeyPress(), which converts a capital Y or N to lower case, ignores any key other than y or n, and counts a wrong key as an error, showing the correction. A correct key calls applyDDA(), which converts clock ticks to seconds, calls updateStreaks() and nextDifficulty(), and stores the returned tier and reason. loadScenario() then asks pickPuzzle() for a term from the new tier, never the one just shown, and writes the tier, topic, term, and prompt to the labels. A prompt longer than 64 characters is split into two balanced lines. Separately, onStep() runs ten times a second, advances the question and session clocks, and calls triggerReport() at tick 1,800. Appendix B reproduces this code.

Table 3.3. Modules of main.py (release v0.4.1).

Module	Lines	Job	Hands off to
Curriculum data	41–73	23 terms in three tiers (10, 6, and 7), each with a prompt, an answer key, and a correction	State manager, through pickPuzzle()
Interface	78–111	Header, eleven game labels, a streak star, and a hidden report panel	The student; written by the state manager
DDA engine and helpers	116–184	Pure decision functions	State manager, as return values
State manager	189–246	showPrompt(), showStreak(), loadScenario(), applyDDA(), and triggerReport()	Engine, data, and interface
Event listeners	251–284	onStep() ten times a second; onKeyPress() on every key	State manager

3.4 Design Diagrams

Figure 3.1 serves as the data flow diagram of the system: each arrow is labelled with the data that moves between modules. Figure 3.2 is the activity diagram of the release process. The system

stores no data, so there is no entity-relationship model. Its single data structure is the curriculum dictionary, keyed by tier, in which each term carries a prompt, an answer key, and a correction.

4. RESULTS AND EVALUATION

This chapter reports what was built and what was measured. Unless stated otherwise, results are for release v0.4.1, the version tested for Unit 7. The code, tests, evaluation scripts, and result files are in the repository.

4.1 Implementation Overview

The product is one file, main.py, of 286 lines. Table 4.1 lists the work by unit. Unit 4 produced the first browser prototype. Unit 5 completed it, rebuilt the core algorithm as testable functions, and found and fixed five defects. Two of them shaped the rules. In v0.2.0 the streak rule was checked first, so three correct answers that each took 20 seconds still promoted the student, and demotion required two errors on the same question, which almost never happens with two keys. The current rules check overload first and count consecutive questions missed on the first try. Unit 6 integrated and evaluated the system and found four defects that no automated test had caught, because the stand-in objects cannot show what the browser draws.

Table 4.1. Implementation milestones by unit.

Unit	Work completed	Evidence
3	Design; first terminal version of one puzzle	Commit e001799
4	Move to CMU Graphics: tiered curriculum, DDA rules, and interface (v0.1.0)	Commit 9483cd2
5	Session timer, streaks, and report (v0.2.0); decision logic rebuilt as pure functions (v0.3.0); 45 unit tests; five defects fixed	Commit 5108a7c; tests/test_dda.py
6	Six integration tests; sandbox recording; performance, simulation, and usability evaluation; deployment plan (v0.4.0)	tests/test_integration.py; eval/
7	Four usability defects fixed (v0.4.1); 80 tests; system and acceptance testing; maintenance plan	tests/test_usability.py; docs/

Integration here means the hand-offs between the five modules in Table 3.3. Seven integration tests, six from Unit 6 and one added in Unit 7, drive the program through its two event handlers. One plays a full three-minute session and compares the report with an independent tally, and another fires 20,000 random events, valid and invalid, and requires that nothing raises an error and that the interface never falls out of step with the engine.

Unit 7 fixed the four defects found in Unit 6 as release v0.4.1 (Figure 4.1). Release v0.4.0 had added only tests and scripts, so the code those defects were found in is the main.py of v0.3.0. Prompts longer than 64 characters are now split into two balanced lines. The header, Errors, and Streak colours were darkened to pass the contrast minimum. The flame character, which the canvas did not draw, was replaced by a star shape that appears from a streak of three. Capital Y and N are accepted. The DDA rules were not changed, so the behavioural results from Unit 6 still apply.

```
158 def streakText(streak):
159     """Streak label text. Plain text only: the canvas does not draw emoji."""
160     return f"Streak: {streak}"
161
162 def showStreakStar(streak):
163     """The star beside the streak label appears from a 3-answer streak onward."""
164     return streak >= STREAK_TO_PROMOTE
165
166 def wrapPrompt(text):
167     """Split a prompt that is too long for one line into two balanced lines.
168     Returns (line1, line2); line2 is empty when the prompt fits on one line."""
169     if len(text) <= MAX_PROMPT_CHARS:
170         return text, ""
171     words = text.split(" ")
172     best = 1
173     for i in range(1, len(words)):
174         left = len(" ".join(words[:i]))
175         bestLeft = len(" ".join(words[:best]))
176         if abs(2 * left - len(text)) < abs(2 * bestLeft - len(text)):
177             best = i
178     return " ".join(words[:best]), " ".join(words[best:])
179
180 def normalizeKey(key):
181     """Treat capital Y and N (Caps Lock or Shift) as y and n."""
182     if key in ('Y', 'N'):
183         return key.lower()
184     return key
```

Figure 4.1. Helper functions added or changed in release v0.4.1 (main.py, lines 158–184).

4.2 Output and Screenshots

Figure 4.2 shows the system in the CS Academy sandbox, with the code on the left and the canvas on the right. To record real outputs I played a complete session of release v0.4.1 in the sandbox on

11 October 2026, in Chrome 154 on Windows. Key presses were sent by a browser automation tool, and a script in the page time-stamped every key press and every change to the canvas. Table 4.2 is the input–output log. Its seconds are measured from the moment each question was drawn, so the on-screen clock, which starts at the key press, can read 0.1 second more.

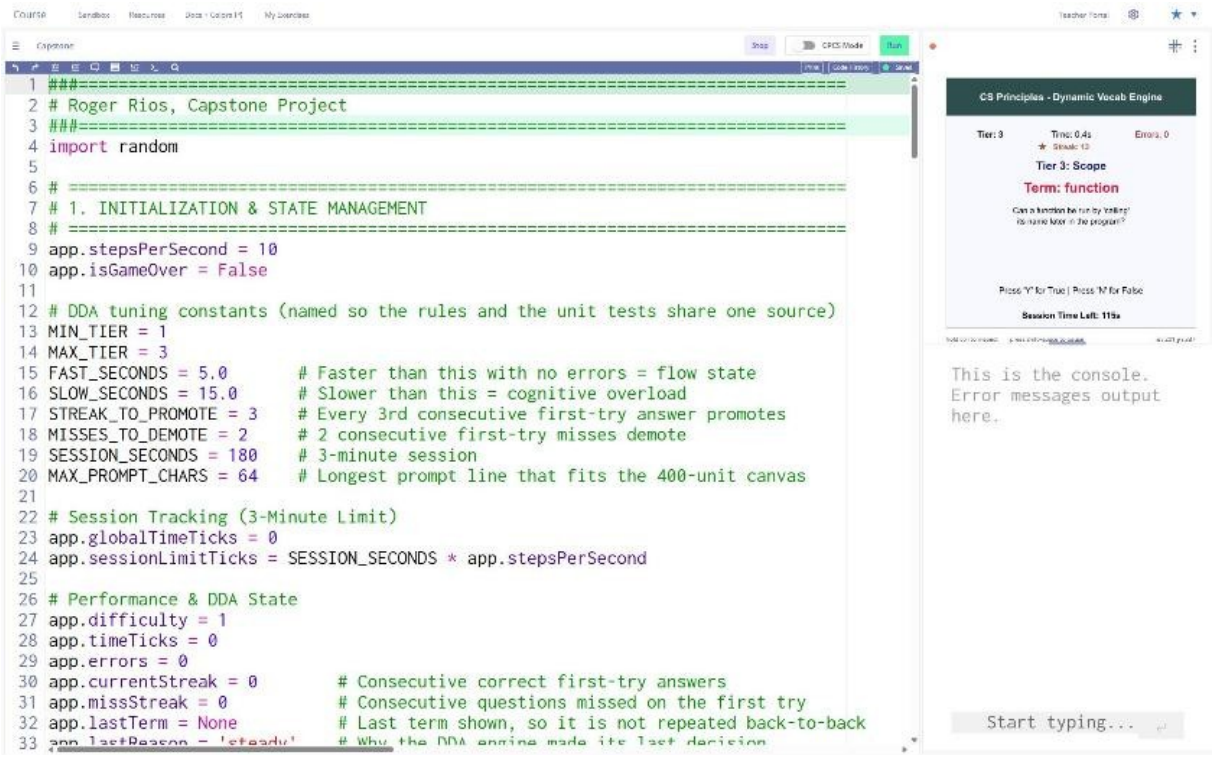


Figure 4.2. The system in the CS Academy sandbox.

Table 4.2. Input–output log of the recorded session.

Q	Term (tier)	Keys	Seconds	Rule	Tier after	Streak after
1	booleans (1)	Y, N	14.5	steady	1	0
2	variable (1)	capital Y	9.5	steady	1	1
3	integer (1)	N	12.1	steady	1	2
4	bug (1)	N	7.9	streak	2	3
5	for-loop (2)	N	10.8	steady	2	4
6	while-loop (2)	N, Y	11.7	steady	2	0
7	for-loop (2)	A, Space, N	13.4	steady	2	1
8	if-elif-else (2)	Y	1.3	flow	3	2
9	arguments (3)	Y	23.7	overload	2	0
10	event (2)	N, Y	9.1	steady	2	0
11	conditional (2)	Y, N	8.6	overload	1	0
12	type (1)	Y	8.3	steady	1	1
13	string (1)	N	10.3	steady	1	2

Every rule appears in the log. In question 1 the wrong key brought up the correction and set the error count to one, and question 4, the third first-try answer in a row, promoted the student to Tier 2, where the star appeared beside the streak (Figure 4.3, a and b). Question 8 was answered in 1.3 seconds and promoted on the flow rule. Question 9 took 23.7 seconds and demoted the student from Tier 3 even though the answer was correct (Figure 4.3, c and d). Questions 10 and 11 were both missed on the first try, and the second miss demoted the student to Tier 1. In question 7 the keys A and Space were pressed first and the screen did not change (Figure 4.6, b). Question 2 was answered with a capital Y, which a script in the page had to send because the automation tool types only lower-case letters. Figure 4.4 plots the tier across the session. Figure 4.5 shows the four prompts that were clipped in the previous release, now drawn in full on two lines; the fourth was captured in a second run.



(a) Wrong key: correction shown, Errors 1

(b) Third first-try answer: Tier 2

(c) Tier 3 term, 23.8 s on the clock

(d) After that answer: back to Tier 2

Figure 4.3. Wrong key, streak promotion, and overload demotion.

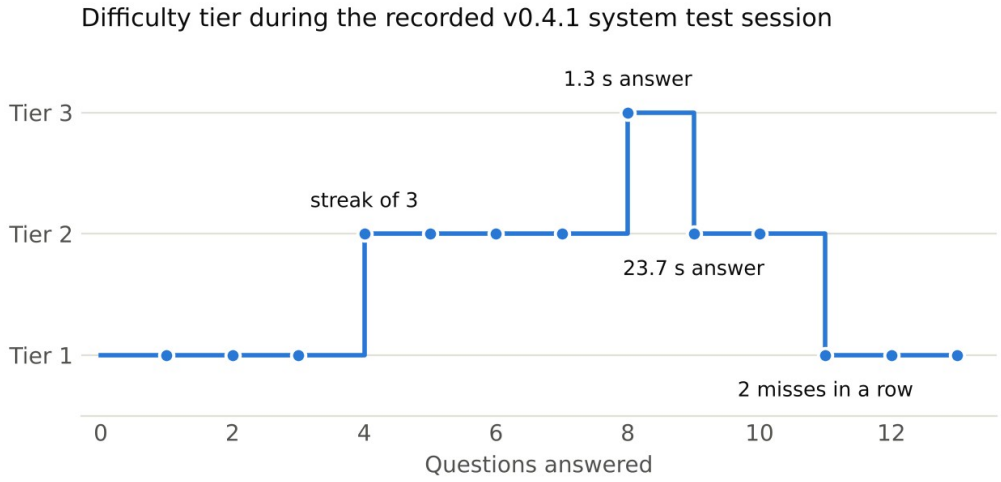
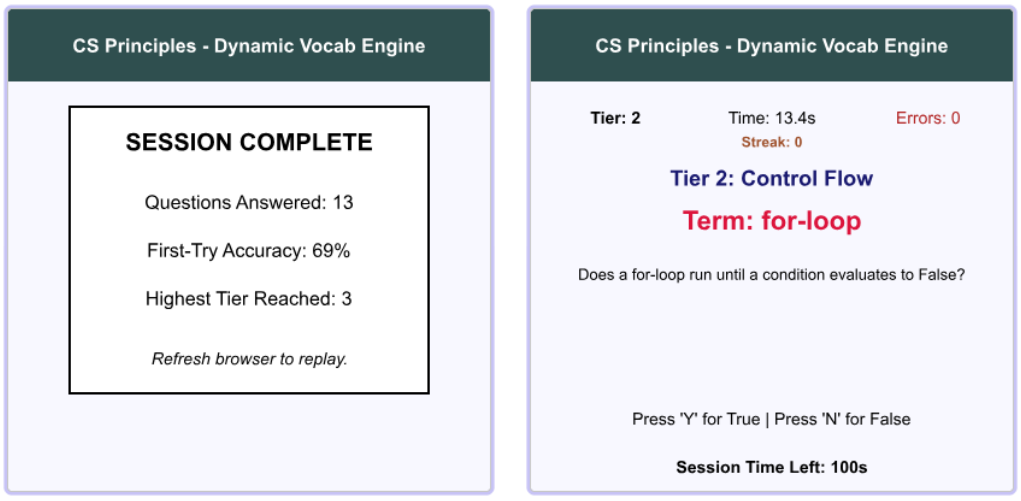


Figure 4.4. Difficulty tier after each answer in the recorded session.



Figure 4.5. Long prompts drawn in full on two lines.

When the countdown reached zero the game labels were hidden and the report appeared (Figure 4.6). It showed 13 questions answered, 69% first-try accuracy, and a highest tier of 3. My own tally is 13 questions with 9 answered correctly on the first try, which is 69% after truncation. As a further check I replayed the logged inputs through the engine functions, and the tier and streak matched the browser after all 13 answers.



(a) End-of-session report (b) After pressing A and Space: no change

Figure 4.6. End-of-session report (a); invalid keys ignored (b).

4.3 Evaluation Metrics

Following LaunchNotes (2023), the evaluation uses both quantitative and qualitative metrics, with at least one for each quality attribute listed by GeeksforGeeks (2025a) and one for the quality of the adaptation, which is the purpose of the product.

4.3.1 Correctness and Regression

Unit 7 testing began with failures. Before changing any code I wrote 19 tests for the four defects and ran them against release v0.3.0, where 11 failed (Figure 4.7). After the fixes, four of the 51 existing tests failed because they still expected the old behaviour, such as the flame character, and were updated. Release v0.4.1 passes all 80 tests: the 51 earlier ones, the 19 usability tests, and ten more for the new functions and for NFR5 (Table 4.3 and Figure 4.8). Together they execute all 184 executable lines of main.py. Coverage shows that lines ran, not that the tests would notice a fault, so a mutation check plants one fault at a time, such as changing > to >= at the 15-second boundary, and runs the suite. Its first run detected 29 of 30 faults. The survivor exposed a missing boundary test for prompt wrapping, and with that test added the suite detects all 30 (Appendix C).

Table 4.3. Test levels and results.

Level	Run against	Cases	Passed	Failed
Usability regression (new tests)	v0.3.0, before the fixes	19	8	11
Earlier suite	Fixed code, before the tests were updated	51	47	4
Unit	v0.4.1	54	54	0
Integration	v0.4.1	7	7	0
Usability regression	v0.4.1	19	19	0
System (sandbox)	v0.4.1	17	17	0
Acceptance	v0.4.1	9	8	1

```
$ pytest -v --no-header tests/test_usability.py          (new tests, run against release v0.3.0)
tests/test_usability.py::test_long_prompt_is_shown_in_full_on_two_lines[return] FAILED [  5%]
tests/test_usability.py::test_long_prompt_is_shown_in_full_on_two_lines[while-loop] FAILED [ 10%]
tests/test_usability.py::test_long_prompt_is_shown_in_full_on_two_lines[if-elif-else] FAILED [ 15%]
tests/test_usability.py::test_long_prompt_is_shown_in_full_on_two_lines[conditional] FAILED [ 21%]
tests/test_usability.py::test_short_prompt_stays_on_one_line PASSED      [ 26%]
tests/test_usability.py::test_every_prompt_line_fits_the_canvas_width FAILED [ 31%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[tierLabel] PASSED [ 36%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[timeLabel] PASSED [ 42%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[errorLabel] FAILED [ 47%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[streakLabel] FAILED [ 52%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[topicLabel] PASSED [ 57%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[promptLabel] PASSED [ 63%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[feedbackLabel] PASSED [ 68%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[instrLabel] PASSED [ 73%]
tests/test_usability.py::test_label_text_meets_wcag_contrast[globalTimerLabel] PASSED [ 78%]
tests/test_usability.py::test_header_title_meets_wcag_contrast FAILED    [ 84%]
tests/test_usability.py::test_streak_of_three_shows_a_marker_the_canvas_can_draw FAILED [ 89%]
tests/test_usability.py::test_streak_marker_disappears_when_the_streak_breaks FAILED [ 94%]
tests/test_usability.py::test_capital_letters_are_accepted_as_answers FAILED [100%]
======================== 11 failed, 8 passed in 0.17s ========================
```

```
$ pytest -q --no-header         (v0.3.0 test suite run against the fixed main.py, before the tests were updated)
.........................F..F..F..........F...                      [100%]
FAILED tests/test_dda.py::test_streak_text_flame_is_valid_unicode - Assertion...
FAILED tests/test_dda.py::test_other_keys_are_ignored - AssertionError: asser...
FAILED tests/test_dda.py::test_three_streak_promotes_and_shows_flame - Assert...
FAILED tests/test_integration.py::test_wrong_key_feedback_comes_from_the_same_curriculum_entry
4 failed, 47 passed in 0.36s
```

Figure 4.7. Failed test cases before the fixes and before the test update.

```
$ pytest tests/test_integration.py -v
tests/test_integration.py::test_key_press_flows_through_every_module PASSED [ 14%]
tests/test_integration.py::test_climb_then_fall_keeps_ui_and_engine_in_step PASSED [ 28%]
tests/test_integration.py::test_wrong_key_feedback_comes_from_the_same_curriculum_entry PASSED [ 42%]
tests/test_integration.py::test_full_session_report_matches_engine_counters PASSED [ 57%]
tests/test_integration.py::test_timer_labels_track_the_step_clock PASSED [ 71%]
tests/test_integration.py::test_random_input_never_breaks_the_system PASSED [ 85%]
tests/test_integration.py::test_new_terms_can_be_added_without_changing_the_engine PASSED [100%]
============================== 7 passed in 0.12s ===============================
```

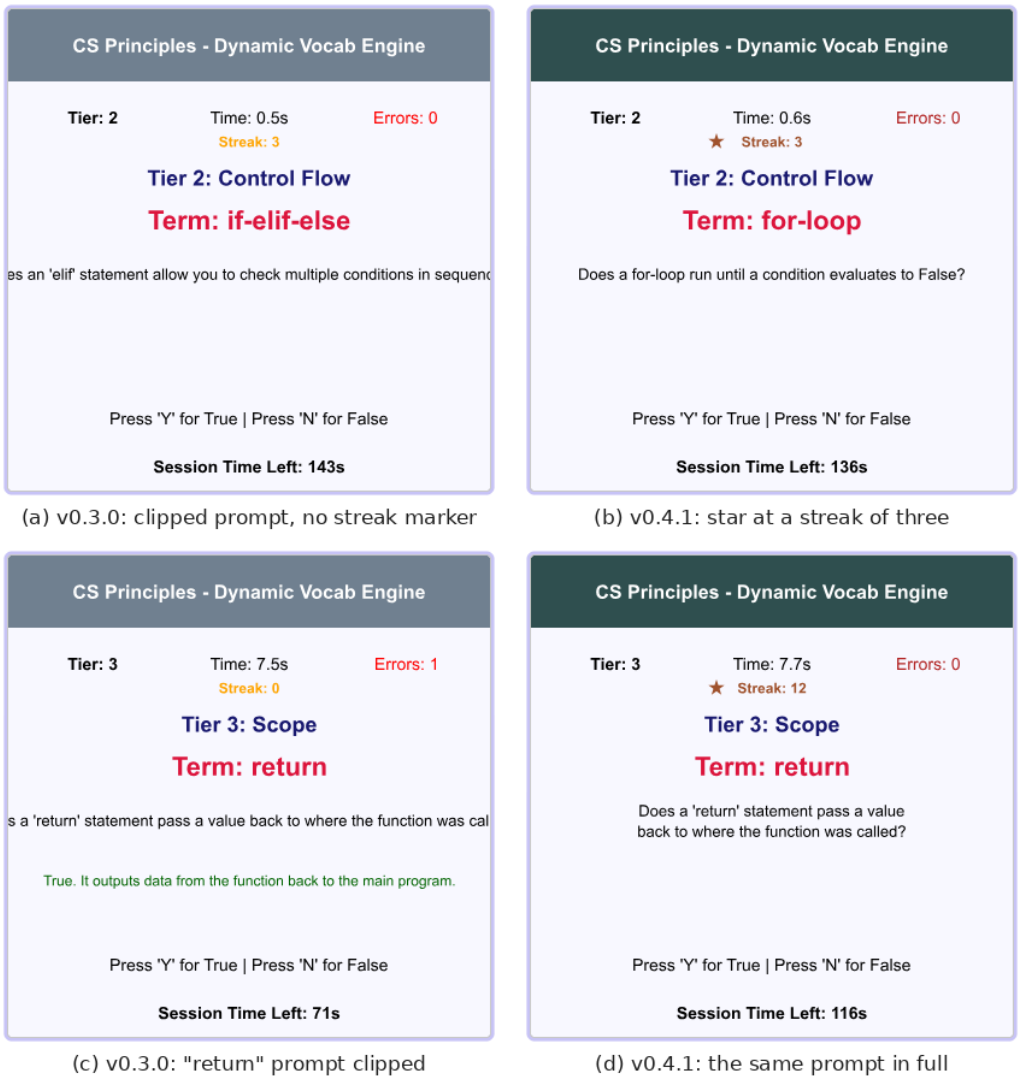
```
$ python tests/coverage_report.py
.......................................................................... [ 90%]
........                                                                 [100%]
80 passed in 2.57s

main.py line coverage (Python trace module)
  executable lines: 184
  executed by tests: 184
  missed lines: none
  coverage         : 100.0%
```

Figure 4.8. Integration tests and the full run with line coverage.

4.3.2 System and Acceptance Testing

The recorded session in Section 4.2, with a second short run to reach the remaining long prompts, was also the system test. All 17 cases passed (Appendix A), including the four that correspond to the defects of the previous release; Figure 4.9 shows two of them before and after. The browser console showed no errors, and a key pressed after the report changed nothing. For acceptance, a script accepts each requirement only if its named tests pass, a static scan of main.py finds no forbidden call where one applies, and the recorded session agrees. Eight of nine requirements were accepted (Figure 4.10). FR3, the manual tier override for teachers, is not implemented.



(a) v0.3.0: clipped prompt, no streak marker

(b) v0.4.1: star at a streak of three

(c) v0.3.0: "return" prompt clipped

(d) v0.4.1: the same prompt in full

Figure 4.9. System test before (v0.3.0) and after (v0.4.1) the fixes.

```
$ python eval/acceptance.py
Acceptance checklist - Dynamic CS Vocabulary Engine v0.4.1

  FR1   ACCEPTED  One term and one True/False question; Y and N keys only
                  4 of 4 tests; sandbox ignored 3 other key presses
  FR2   ACCEPTED  Tier raised, held or lowered from time and errors
                  17 of 17 tests; sandbox matched the rules on 13 of 13 answers (flow, overload, steady, streak)
  FR3   NOT MET   Teacher can set the tier by hand
                  no manual tier override in main.py (deferred; listed as open in the README)
  FR4   ACCEPTED  Report at the end; nothing stored between sessions
                  3 of 3 tests; sandbox report equals the hand tally; no file calls
  NFR1  ACCEPTED  Security: restricted input, no dynamic execution, no files
                  2 of 2 tests; no eval, exec, open or input call; only import is random
  NFR2  ACCEPTED  Usability: labels fully visible; contrast 4.5:1
                  19 of 19 tests; 12 of 12 labels meet the contrast minimum; 0 texts too wide
  NFR3  ACCEPTED  Performance: response within 1 s (target 0.1 s)
                  slowest of 36 sandbox responses 60.7 ms (limit 1,000; target 100)
  NFR4  ACCEPTED  Reliability: unexpected and late input ignored
                  3 of 3 tests; 1 key press after the report changed nothing
  NFR5  ACCEPTED  Scalability: terms are data
                  2 of 2 tests; a term added to the data is served with no change to the engine

  accepted: 8 of 9 requirements
  results written to eval/results/acceptance.json
```

Figure 4.10. Output of the acceptance checklist.

4.3.3 Performance and Reliability

Performance was measured in two places. In the sandbox, the page script sampled the canvas about 250 times a second, a resolution of about 4 ms, and recorded the delay between each key press and the next change in the question area, across 36 presses in two sessions, together with every clock tick of one full session. Outside the browser, a benchmark timed the program's logic in CPython 3.13 with the stand-in graphics objects. Table 4.4 gives the results, and Figure 4.11 shows the timing output; a single call to nextDifficulty() takes about 0.2 µs. The sandbox figures are close to those of release v0.3.0 (median 55.9 ms; session length 180.104 s), so the fixes cost nothing measurable.

Table 4.4. Performance and reliability results.

Metric	Result	Target
Key press to display, sandbox (n = 36)	Median 54.8 ms; range 6.6–60.7 ms	Under 1 s; target 0.1 s
Step interval, sandbox (n = 1,793)	Mean 100.28 ms; 99.1% within 10 ms of target	100 ms
Countdown interval, sandbox (n = 179)	Mean 1,000.04 ms	1,000 ms
Session length, Run click to report	180.155 s (+0.09%)	180 s
Start-up, Run click to first frame	165 ms	None set
Answer cycle, CPython (n = 200,000)	Median 3.1 µs; 99th percentile 21.6 µs; slowest 3.2 ms	Under 1 s
Memory, CPython	30.8 KiB held after loading; 920.1 KiB peak while compiling	None set

```
$ python eval/browser_timing.py
Dynamic CS Vocabulary Engine v0.4.1 - timing in the CS Academy sandbox
  Chrome 154 on Windows, canvas 400x400, background tab (visibilityState hidden)

Key press to updated display
  session 1 (scripted play)   n=16  median 55.2  mean 50.0  min 6.6  max 60.7 ms
  session 2 (20 rapid keys)   n=20  median 54.4  mean 54.5  min 53.3  max 55.6 ms
  both sessions               n=36  median 54.8  mean 52.5  min 6.6  max 60.7 ms
  presses slower than 100 ms  0 of 36
  presses with no display change (invalid key, or after the session ended): 3 ['a', 'space', 'y']
  capital letters accepted (script-dispatched, left out of the timing): 1 ['Y']

Clock accuracy over one full session
  step interval (target 100 ms)       mean 100.28  median 100.0  p95 101.3  max 200.5 ms  (n=1793)
  steps within 10 ms of target        1776 of 1793 (99.1%)
  countdown interval (target 1000 ms) mean 1000.04  sd 1.1  min 996  max 1007 ms  (n=179)
  Run click to report (target 180 s)  180.155 s  (+0.09%)
  first frame to report               179.990 s
  start-up (Run click to first frame) 165 ms
```

Figure 4.11. Timing recorded in the CS Academy sandbox.

4.3.4 Adaptation Quality

Two comparisons were run between v0.2.0 and the current rules. The first replays fixed scripted sessions. A student who misses twelve questions in a row stays at Tier 3 under v0.2.0 and falls to Tier 1 within four questions under the current rules, and back-to-back repeats of the same term fell from 324 in 2,000 draws to none. The second uses simulated learners. Each has a mastery tier. On terms at or below that tier the learner answers correctly on the first try 90% of the time, in 3 to 12

seconds. On harder terms the learner is right half the time, which is chance on a True/False item, and takes 6 to 20 seconds. These values are my assumptions, not measurements of students. Table 4.5 reports 1,000 three-minute sessions per row.

Table 4.5. Simulated learners: questions served relative to the learner's tier.

Learner	Version	At tier	Too hard	Too easy	Tier changes per session
Tier 1	v0.2.0	47.6%	52.4%	0.0%	5.57
Tier 1	Current	55.0%	45.0%	0.0%	6.27
Tier 2	v0.2.0	41.9%	41.3%	16.9%	5.73
Tier 2	Current	47.5%	34.9%	17.6%	6.32
Tier 3	v0.2.0	75.7%	0.0%	24.3%	2.00
Tier 3	Current	73.7%	0.0%	26.3%	2.35

4.3.5 Usability

For the qualitative metric I reviewed the running interface against Nielsen's (2024) ten usability heuristics and rated each problem on his severity scale (Nielsen, 1994). Two measurements support the review. A contrast check computes the ratio between each label's text colour and its background and compares it with the minimum of the World Wide Web Consortium (2026): 4.5:1 for normal text and 3:1 for large text. A width check measures every line of every prompt against the 400-unit canvas. Table 4.6 lists the findings and their status after Unit 7. The two major problems and two smaller ones are fixed: all 12 label styles checked now meet the contrast minimum, and no prompt or correction is wider than the canvas. Four lesser problems remain open.

Table 4.6. Heuristic review findings and their status in release v0.4.1.

Finding	Severity (0–4)	Status
Status, corrections, and key instructions are always visible	–	Strength
Four of 23 prompts clipped at both edges	3	Fixed: two-line wrapping
Streak (1.87:1), Errors (3.78:1), and header (4.05:1) text below 4.5:1	3	Fixed: 5.31:1, 6.32:1, and 8.93:1
Capital Y and N ignored	2	Fixed
Streak flame not drawn by the sandbox	1	Fixed: star shape
A tier change is not announced or explained	2	Open
No pause, restart, or quit inside the app	2	Open
Tiers, streaks, and adaptation are not explained to the student	2	Open
Prompts are yes/no questions, but the keys are labelled True and False	1	Open

5. DISCUSSION

5.1 Interpretation of Results

Performance. The system meets its performance requirement by a wide margin. NFR3 required the next question within one second. The slowest of 36 responses in the sandbox was 60.7 ms, and Figure 5.1 shows that every one fell below the 0.1 s limit under which, according to Nielsen (1993), a system feels as if it reacts instantaneously. The engine accounts for almost none of that time: the complete answer cycle takes about 3 µs in CPython, so nearly all of the 55 ms is spent in the sandbox's own event handling and drawing. Two presses registered in about 7 ms. I could not determine why from outside the sandbox, and the values are reported as measured.

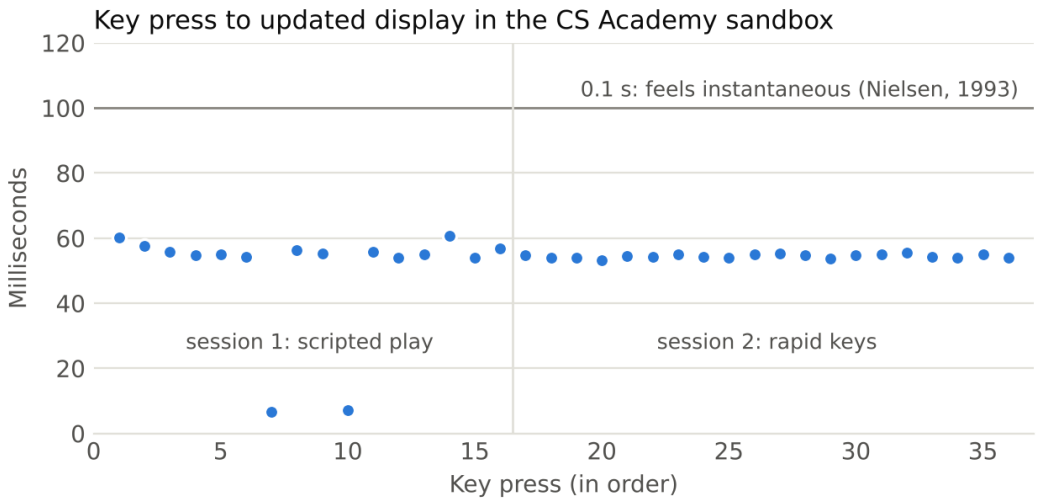


Figure 5.1. Key press to updated display for all 36 measured presses.

Reliability of the clock. The DDA rules depend on time, so an inaccurate clock would mean unfair decisions. The step interval averaged 100.28 ms, 99.1% of steps were within 10 ms of the target, and the session ended 180.155 s after the Run click. The clock is accurate enough for thresholds of 5 and 15 seconds.

Correctness. Full line coverage, a clean mutation check, and a passing system test are good evidence that the code does what the rules say. They do not show that the rules are the right ones.

Adaptation quality. Figure 5.2 shows the most important result of the evaluation and the least flattering. The Unit 5 changes helped the learners they were aimed at: simulated Tier 1 learners received questions at their own tier 55.0% of the time instead of 47.6%, and Tier 2 learners 47.5% instead of 41.9%. Yet even under the current rules a Tier 1 learner spends 45% of the session on terms that are too hard. A single fast correct answer promotes, and every third correct answer promotes again, so the engine keeps probing upward and being knocked back, changing tier about six times in a session of 18 questions. The design phase anticipated this “difficulty whiplash” and proposed a moving average over the last three attempts. That smoothing was never built, and the simulation shows the cost.

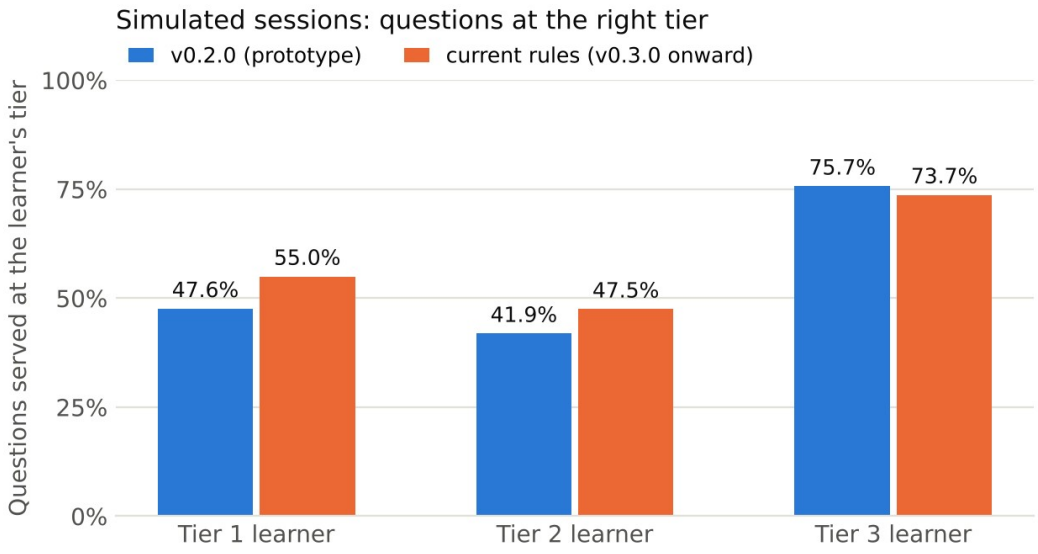


Figure 5.2. Simulated sessions: questions at the learner’s tier, by version.

Testing in levels. The Unit 7 results show why testing has to reach the real runtime. The unit tests gave full coverage in Unit 5 and confirmed that the streak label contained a flame character and that

every prompt was loaded. Only the running canvas showed that the flame was not drawn and that four prompts were cut off. Each defect now has a test that failed before the fix and passes after it, so it cannot return unnoticed.

Usability. The review found a screen that communicates status well and corrects mistakes clearly. The two major problems are gone, and the automated width and contrast tests now guard against their return when terms are added. The open findings share a cause: the student is not told how the adaptation works and is given no control over the session, which is also where the transparency promised in the proposal is still unmet.

5.2 Comparison with Literature Findings

This project sits in the mainstream that Víteková et al. (2026) describe: a rule-based engine driven by unobtrusive performance measures. Their warning also applies. They found that outcome measures differ widely and that samples are small, and my results should be read with the same caution: the engine works as specified, but nothing here shows that it improves learning or engagement. Seyderhelm and Blackmore (2024) show what such evidence looks like, with 52 participants divided between an adaptive and a linear version of a training game. Their system measured cognitive load. Mine infers it from answer time and errors, and the simulation suggests that these indicators are noisy when read one answer at a time. A student who knows a term can still take 16 seconds, and a student who is guessing is right half the time on a True/False item. By the standard of the field this project is at an earlier stage: verification is complete and validation is still to come, the distinction that Summers (2020) draws.

On the engineering side the results agree with the testing literature. Amazon Web Services (n.d.) notes that unit tests run in isolation with stubs, and GeeksforGeeks (2026) that unit tests cannot

detect integration or usability problems. Both were borne out: the stand-ins made full coverage possible and hid the rendering defects, which appeared only when the whole system was run in the sandbox, the level of testing that Hrupa (2024) places before acceptance. Wells (2025) argues that measuring quality replaces subjective debate with evidence, and the contrast figures illustrate the point: “the orange looks pale” became “1.87:1 against a required 4.5:1”.

5.3 Implications and Limitations

For teachers, the implication is practical. An adaptive practice tool can be delivered as a single file that runs in a browser the school already has, responds without perceptible delay, and stores no student data. For the project, the implication is that the engineering is ahead of the pedagogy. Table 5.1 compares the requirements with what was delivered.

Table 5.1. Requirements and their status in release v0.4.1.

ID	Status	Evidence
FR1	Met	System test; keys A and Space ignored in the sandbox
FR2	Met	All four rules observed; replay matched 13 of 13 answers
FR3	Not met	No manual override; listed in the README as open
FR4	Met	Report equals the hand tally; no file calls in main.py
NFR1	Met	Static scan: no eval, exec, open, or input; one import (random)
NFR2	Met	19 usability tests; 12 of 12 labels pass; no text too wide
NFR3	Met	Slowest of 36 responses 60.7 ms
NFR4	Met	20,000 random events without an error; late key ignored
NFR5	Met	Integration test adds a term without changing the engine

Deployment. Sumo Logic (n.d.) defines deployment as the steps that make software available to its users. Students will use the hosted sandbox, the only option that needs no installation on school devices, and the desktop CMU Graphics package (CMU CS Academy, 2026) is the untested fallback for labs without internet. A release is a merge into main, an annotated tag, a hash check,

and a paste into the sandbox, and rolling back means pasting the file from the previous tag. Configuration management keeps this arrangement reliable (Bennett, 2024). The configuration items are main.py, the tests, the evaluation scripts, the result files, and the documents. Every Git tag is a baseline, and the hash comparison is the audit, which matters because the deployed copy can drift from the repository without anyone noticing. The time, streak, and miss thresholds, the session length, and the clock rate are named constants at the top of main.py, so changing one is a one-line commit that can be reviewed, tested, and reverted. A student device needs only a current browser, a keyboard, internet access, and a CS Academy account.

Maintenance. Table 5.2 summarizes the maintenance plan by the four types that Thales (n.d.) names. Corrective and preventive work matter most now, and adaptive work matters most over time, because the program depends on a third-party sandbox that can change without notice. Bug reports and enhancement requests are GitHub issues, releases follow semantic versioning, and each release is recorded in a changelog in the same commit as the code, in line with Brown's (2025) advice to write documentation close to the time of development. The largest risk after deployment is an outage or interface change at CS Academy, which would stop every student at once. The other risks are a change, including AI-assisted code, that breaks a rule, which the tests and the roll-back procedure contain; an engine that misjudges a student, mitigated by named thresholds and by simulating any change before release; and a wrong answer key, mitigated by a content review each term. Performance is compared each term with the v0.4.1 baseline in Table 4.4.

Table 5.2. Maintenance plan.

Type	Planned work	When
Corrective	Issue, failing test, fix, patch release, as in v0.4.1	As reported
Adaptive	Read CS Academy and browser release notes; smoke test in the sandbox	Monthly; weekly in term
Perfective	Teacher override; reasons for tier changes; steadier promotion; more Tier 2 terms	Minor releases
Preventive	Tests, coverage, and mutation check; hash audit of the sandbox copy	Every push; every release

Limitations. First, no students have used the system. Second, the simulated learners rest on my assumptions about accuracy and speed, so the direction of the comparison between versions is more trustworthy than the absolute values. Third, the usability review had a single evaluator; Moran and Gordon (2023) recommend three to five, and Nielsen (1994) considers severity ratings from one evaluator unreliable. Fourth, the browser timing comes from one machine and one browser, with key presses sent by an automation tool to a background tab, and the capital letter was sent by a script and not from a physical keyboard. A school Chromebook may be slower. Fifth, the benchmark ran in CPython with stand-in graphics objects, not in the sandbox's own Python runtime. Sixth, the content is small: 23 terms, with only six in Tier 2.

5.4 Future Work

The results point to research that the prototype cannot yet answer. The first question is whether the tool helps. A small classroom trial comparing the adaptive engine with a fixed sequence, measuring vocabulary gain and using a short engagement survey, would follow the kind of comparison Seyderhelm and Blackmore (2024) made and would respond to the call by Víteková et al. (2026) for larger and more consistent evaluations. The second concerns the signal. A format with a 50% guess rate gives the engine weak evidence, and it would be worth testing whether four-

option items make simple rule-based adjustment markedly more accurate. The third concerns the rules: the simulation can compare a moving average, a pause on promotion after a demotion, and a requirement for speed and a streak together before any student sees them. Section 6.4 lists the engineering steps.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of Main Findings

This project set out to build an adaptive vocabulary trainer for introductory computer science that a school could run without installing anything. The result is a single Python file running in the CMU CS Academy sandbox, with a rule-based DDA engine that moves a student among three tiers according to answer time and errors.

The evaluation supports four findings. The system is correct against its specification: 80 tests pass, they cover every executable line, they detect all 30 planted faults, and the released file passed all 17 system test cases in the sandbox. It is fast and its clock is accurate: the display updates about 55 ms after a key press, and a three-minute session ran 0.09% long. Its interface now meets its usability requirement, after the four defects that only the real sandbox revealed were fixed. Its adaptation is better than the prototype's but still unstable, because the promotion rules react to single answers. The remaining risk is therefore educational, not technical.

6.2 Contributions

The project makes three modest contributions. It shows that a DDA engine can be written for a constrained single-file classroom environment and still be fully tested, by separating pure decision functions from the graphics layer. It provides a reusable evaluation kit for such a program: integration tests through the event handlers, a mutation check, a simulation of synthetic learners, automated contrast and width checks, a scripted acceptance checklist, and a method for timing a canvas application from inside the page. And it documents a concrete weakness of single-answer promotion rules on True/False items, with numbers against which a later version can be judged.

6.3 Lessons Learned

The first lesson is that passing tests are not a working screen. Full coverage with stand-in objects said nothing about what the sandbox would draw, so the release procedure now includes a system test in the real runtime. The second concerns configuration. The deployed copy lives in the sandbox editor, outside version control, and during Unit 6 I found that the repository had fallen behind it. Comparing hashes is a cheap audit, and the repository, not the sandbox, is now where changes are made first. The third is to measure before believing: I expected the Unit 5 rules to settle learners at their level, and the simulation showed that they oscillate. The fourth is that scope should follow the user's environment. Moving from a terminal to the browser cost the typed answers and stored progress of the proposal, and it is the reason the tool can be used at all.

6.4 Next Steps

The next steps, in priority order, are to build the teacher's manual override (FR3) and show the student why the tier changed; to add a restart key; to compare promotion rules that use more than one answer and keep whichever raises the share of on-level questions without slowing a steady learner; to extend the curriculum, starting with Tier 2; and to test the offline desktop package and the timing on a school Chromebook. A classroom trial, with a second evaluator repeating the heuristic review, would then supply the evidence this report lacks: whether three minutes with the tool helps a student learn.

UNIVERSITY OF THE PEOPLE

REFERENCES

Amazon Web Services. (n.d.). *What is unit testing?* https://aws.amazon.com/what-is/unit-testing/

Bennett, L. (2024, August 13). *Software configuration management in software engineering*. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

Brown, J. (2025, December 2). *Knowledge software documentation best practices [with examples]*. Helpjuice. https://helpjuice.com/blog/software-documentation

BrowserStack. (2025, May 29). *What is system testing? (Examples, use cases, types)*. https://www.browserstack.com/guide/what-is-system-testing

CMU CS Academy. (2026, September 30). *cmu-graphics* (Version 2.0.5) [Computer software]. Python Package Index. https://pypi.org/project/cmu-graphics/

Davies, S. (2024). *Blueprints: Creating, describing, and implementing designs for larger-scale software projects* (Version 2.5). University of Mary Washington. https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis

GeeksforGeeks. (2025a, July 15). *Measuring software quality using quality metrics*. https://www.geeksforgeeks.org/software-engineering/measuring-software-quality-using-quality-metrics/

GeeksforGeeks. (2025b, July 22). *System testing – Software testing*. https://www.geeksforgeeks.org/software-testing/system-testing/

GeeksforGeeks. (2025c, September 29). *Software maintenance – Software engineering*. https://www.geeksforgeeks.org/software-engineering/software-engineering-software-maintenance/

GeeksforGeeks. (2026, June 24). *Unit testing - Software testing*. https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/

GitHub. (n.d.). *Understanding GitHub Actions*. GitHub Docs. https://docs.github.com/en/actions/get-started/understand-github-actions

Google. (2026). *Gemini* [Large language model]. https://gemini.google.com/

Google Antigravity Team. (2025, November 20). *Build with Google Antigravity, our new agentic development platform*. Google for Developers Blog. https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/

Hrupa, A. (2024, March 19). *What is system testing in software testing*. Testfort. https://testfort.com/blog/what-is-system-testing-in-software-testing

LaunchNotes. (2023, September 1). *Qualitative vs quantitative metrics: A comprehensive comparison*. https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison

Letaw, L. (2024). *Handbook of software engineering methods* [Open textbook]. Oregon State University. https://open.oregonstate.education/setextbook/

Moran, K., & Gordon, K. (2023, June 25). *How to conduct a heuristic evaluation*. Nielsen Norman Group. https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/

Nielsen, J. (1993, January 1). *Response times: The 3 important limits*. Nielsen Norman Group. https://www.nngroup.com/articles/response-times-3-important-limits/

Nielsen, J. (1994, November 1). *Severity ratings for usability problems*. Nielsen Norman Group. https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/

Nielsen, J. (2024, January 30). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Oguz, A. (2022). *Project management* (2nd ed.). Pressbooks. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

Seyderhelm, A., & Blackmore, K. (2024). Dynamic adaptive surveillance training in a virtual environment using real-time cognitive load and performance. *International Journal of Serious Games, 11*(3), 109–133. https://doi.org/10.17083/ijsg.v11i3.733

Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publishers.

Sumo Logic. (n.d.). *What is software deployment?* https://www.sumologic.com/glossary/software-development/

Thales. (n.d.). *What is a software maintenance process? 4 types of software maintenance*. https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance

Víteková, L., Eichhorn, C., Pirker, J., & Plecher, D. A. (2026). Dynamic difficulty adjustment in serious games: A literature review. *Information, 17*(1), Article 96. https://doi.org/10.3390/info17010096

Walker, V. (2022, March 16). *14 software architecture design patterns to know*. Red Hat. https://www.redhat.com/en/blog/14-software-architecture-patterns

Wells, J. (2025, April 16). *Top 12 software quality metrics to measure and why*. LinearB. https://linearb.io/blog/software-quality-metrics

World Wide Web Consortium. (2026, September 6). *Understanding success criterion 1.4.3: Contrast (minimum)*. https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

UNIVERSITY OF THE PEOPLE

APPENDICES

Appendix A: System Test Cases

Table A.1. System test cases run in the CS Academy sandbox on release v0.4.1.

ID	Case	Expected result	Result
ST-01	Press Run	First Tier 1 question appears	Pass (165 ms)
ST-02	Wrong key	Correction shown; Errors 1; same question	Pass
ST-03	Right key	Next question within 1 s	Pass (slowest 60.7 ms)
ST-04	Third first-try answer in a row	Tier up; star beside “Streak: 3”	Pass
ST-05	Wrong key during a streak	Streak 0; star removed	Pass
ST-06	First-try answer in under 5 s	Tier up	Pass (1.3 s)
ST-07	Correct answer after 15 s	Tier down	Pass (23.7 s)
ST-08	Two first-try misses in a row	Tier down	Pass
ST-09	Keys A and Space	No change	Pass
ST-10	Capital Y (sent by a script)	Accepted as True	Pass
ST-11	Long prompts	Whole prompt visible on two lines	Pass (3 in the session; 2 in a second run)
ST-12	Header, Errors, and Streak colours	Drawn as coded; at least 4.5:1	Pass
ST-13	Session left to run	Report after 180 s	Pass (180.155 s)
ST-14	Report against tally	Same counts, accuracy, and tier	Pass (13, 69%, 3)
ST-15	Key after the report	No change	Pass
ST-16	Clock steps	100 ms apart	Pass (mean 100.28 ms)
ST-17	Browser console	No error messages	Pass

Appendix B: Code Listings

```
200 def loadScenario():
201     app.timeTicks = 0
202     app.errors = 0
203     puzzle = pickPuzzle(app.curriculum[app.difficulty], app.lastTerm)
204     app.lastTerm = puzzle["term"]
205 
206     tierLabel.value = f"Tier: {app.difficulty}"
207     topicLabel.value = app.topics[app.difficulty]
208     termLabel.value = f"Term: {puzzle['term']}"
209     showPrompt(puzzle["prompt"])
210 
211     errorLabel.value = "Errors: 0"
212     showStreak()
213 
214     app.correctKey = puzzle["ans"]
215     app.currentCorrection = puzzle["correction"]
216     feedbackLabel.value = ""
217 
218 def applyDDA():
219     time_taken = app.timeTicks / app.stepsPerSecond
220 
221     app.currentStreak, app.missStreak = updateStreaks(app.currentStreak, app.missStreak, app.errors)
222     app.difficulty, app.lastReason = nextDifficulty(
223         app.difficulty, app.currentStreak, app.missStreak, time_taken, app.errors)
224 
225     # An overload decision starts the student fresh at the easier tier
226     if app.lastReason == 'overload':
227         app.currentStreak = 0
228         app.missStreak = 0
229 
230     # Track highest tier for the final report
231     if app.difficulty > app.highestTier:
232         app.highestTier = app.difficulty
233 
234     loadScenario()
```

Figure B.1. State manager: loadScenario() and applyDDA() (lines 200–234).

```
251 def onStep():
252     if app.isGameOver:
253         return
254 
255     app.timeTicks += 1
256     app.globalTimeTicks += 1
257 
258     timeLabel.value = f"Time: {app.timeTicks / app.stepsPerSecond}s"
259 
260     # Update global countdown
261     seconds_remaining = secondsRemaining(app.globalTimeTicks, app.stepsPerSecond)
262     globalTimerLabel.value = f"Session Time Left: {seconds_remaining}s"
263 
264     if app.globalTimeTicks >= app.sessionLimitTicks:
265         triggerReport()
266 
267 def onKeyPress(key):
268     # Only y and n (in either case) are accepted; every other key is ignored
269     key = normalizeKey(key)
270     if app.isGameOver or key not in ['y', 'n']:
271         return
272 
273     if key == app.correctKey:
274         if app.errors == 0:
275             app.firstTryCorrect += 1
276 
277         app.totalQuestions += 1
278         applyDDA()    # Streaks and the tier are updated inside the DDA engine
279     else:
280         app.errors += 1
281         app.currentStreak = 0 # Break streak on error
282         errorLabel.value = f"Errors: {app.errors}"
283         showStreak()
284         feedbackLabel.value = app.currentCorrection
```

Figure B.2. Event listeners: onStep() and onKeyPress() (lines 251–284).

Appendix C: Evaluation Output

```
$ python eval/mutation_check.py
Mutation check: one planted fault per run
   1  detected  seconds > SLOW_SECONDS
   2  detected  errors >= 2 or
   3  detected  missStreak >= MISSES_TO_DEMOTE
   4  detected  max(MIN_TIER, tier - 1), 'overload'
   5  detected  streak > 0 and streak % STREAK_TO_PROMOTE == 0
   6  detected  min(MAX_TIER, tier + 1), 'streak'
   7  detected  seconds < FAST_SECONDS
   8  detected  errors == 0 and seconds < FAST_SECONDS
   9  detected  return streak + 1, 0
  10  detected  return 0, missStreak + 1
  11  detected  if p["term"] != lastTerm
  12  detected  int(firstTryCorrect * 100 / totalQuestions)
  13  detected  SESSION_SECONDS - globalTicks // stepsPerSecond
  14  detected  return streak >= STREAK_TO_PROMOTE
  15  detected  if app.lastReason == 'overload':
  16  detected  if app.difficulty > app.highestTier:
  17  detected  app.globalTimeTicks >= app.sessionLimitTicks
  18  detected  if app.isGameOver or key not in ['y', 'n']:
  19  detected  if app.errors == 0:
  20  detected  app.currentStreak = 0 # Break streak on error
  21  detected  if app.isGameOver:
  22  detected  feedbackLabel.value = app.currentCorrection
  23  detected  feedbackLabel.value = ""
  24  detected  app.gameUI.visible = False
  25  detected  if len(text) <= MAX_PROMPT_CHARS:
  26  detected  if abs(2 * left - len(text)) < abs(2 * bestLeft - le
  27  detected  promptLabel2.value = line2
  28  detected  if key in ('Y', 'N'):
  29  detected  streakStar.visible = showStreakStar(app.currentStrea
  30  detected  fill='fireBrick'
  faults detected: 30 of 30
```

Figure C.1. Mutation check: 30 of 30 planted faults detected.

```
$ python eval/benchmark.py
Dynamic CS Vocabulary Engine - engine benchmark
  CPython 3.13.16 on Linux x86_64 (CPython with graphics stand-ins; excludes browser drawing)

  nextDifficulty()               206.2 ns mean   (999,864 calls)
  answer cycle (key -> next)     3.139 us median  p95 11.527  p99 21.619  max 3152.5  (n=200,000)
  answer-cycle throughput       179,565 per second
  onStep() tick                  0.791 us median  p95 0.912  p99 1.467  max 89.5  (n=200,000)
  load main.py                   1.688 ms median  p95 2.985  (n=200)
  peak heap while loading        920.1 KiB   (compiling the source)
  heap held after load            30.8 KiB
  peak heap, full session         32.1 KiB   (29 questions, 31.3 KiB at the end)
  main.py source size            14,869 bytes

  NFR: a decision in under 1 s. Slowest answer cycle observed: 3.153 ms
  results written to eval/results/benchmark.json
```

Figure C.2. Engine benchmark output.

Appendix D: Project Schedule

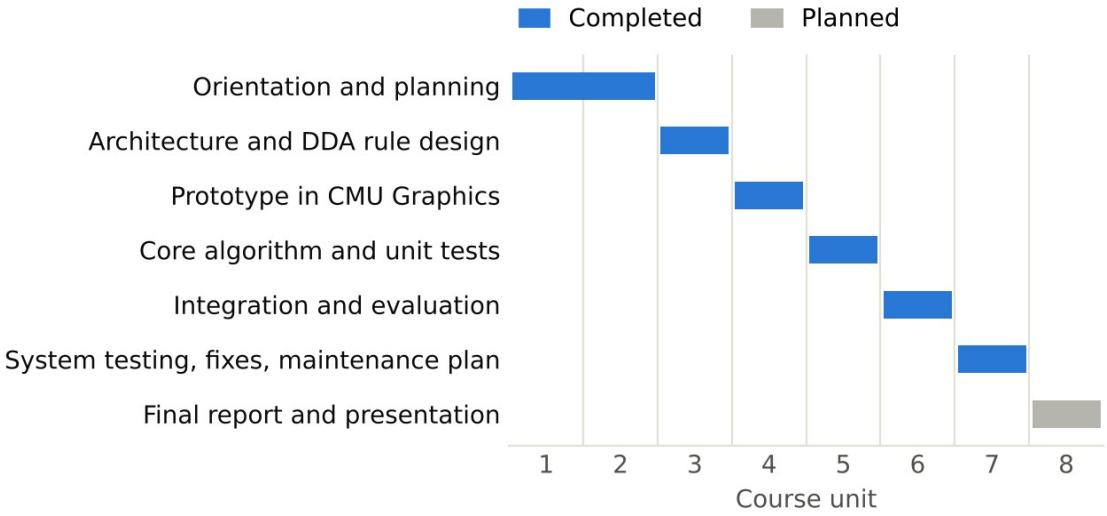


Figure D.1. Gantt chart of the project by course unit (3 September to 28 October 2026).